



Universidad
Manuela Beltrán

Acreditada en Alta Calidad Multicampus

Guía 1 de Laboratorio - Definición del Proyecto

Juan Camilo Cespedes Correal

Juan Camilo Zabala Ceron

David Felipe Moreno García

Brayhan Alexander Suárez Merchan

Universidad Manuela Beltrán

Carlos Eduardo Mujica Reyes

Arquitectura de Software

2026

Definición de Proyecto ERP

1. Evaluar Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Técnico	Se puede empezar simple (módulos básicos: inventario, ventas, gastos, nomina) e ir creciendo; no requiere integraciones complejas con sistemas externos al inicio	Requiere mantenimiento continuo (actualizaciones, backups, soporte); el equipo debe aprender a usarlo
Relevancia del Problema	El negocio maneja muchas categorías de producto con rotación distinta (una gaseosa rota rápido, un licor puede quedarse semanas), lo cual hoy probablemente se controla manualmente o en Excel, generando errores de stock y pérdidas	Si el proceso actual "funciona" aunque sea informal, puede haber resistencia al cambio por parte de quien lleva las cuentas hoy
Valor Para el Usuario Final	Permite saber en tiempo real qué hay en bodega, qué se está por acabar, cuánto se gasta y cuánto se gana por categoría; reduce pérdidas por vencimiento o robo hormiga	Curva de aprendizaje inicial para los empleados que hoy no usan sistemas digitales
Financiero	Bajo costo de desarrollo (proyecto académico/propio) vs. comprar un ERP comercial (Siigo, Alegra, SAP Business One) que cobra licencia mensual	El desarrollo toma tiempo y no genera valor inmediato hasta que esté terminado y en uso
Control del Negocio	Al ser un desarrollo propio, se ajusta 100% a cómo trabaja el negocio (venta por unidad, por paquete, por caja; productos con IVA vs. sin IVA como cigarrillos y licores)	Un ERP comercial ya viene probado y con soporte; el propio requiere que alguien lo mantenga después de terminado el curso

2. Definición general del proyecto

A. Definición del problema

La empresa Víveres y Licores la 74 dedicada a la venta de víveres, bebidas, licores, cigarrillos, actualmente no cuenta con un sistema centralizado para el control de inventario, costos, gastos y nómina de empleados. Esto genera:

- Desconocimiento del stock real disponible en un momento dado.
- Dificultad para identificar qué productos generan más pérdida (vencimiento, robo, mal conteo).
- Falta de visibilidad sobre los gastos totales del negocio (servicios, nómina, compras) frente a los ingresos.
- Decisiones de compra a proveedores basadas en intuición y no en datos.

B. Solución ofrecida

Un sistema ERP (Enterprise Resource Planning) que centralice:

- **Módulo de inventario:** registro de entradas y salidas de productos por categoría (víveres, licores, cigarrillos, gaseosas/jugos, mecatos), con alertas de stock mínimo.
- **Módulo de costos y ventas:** registro de precio de compra vs. precio de venta, cálculo de margen por producto/categoría.
- **Módulo de empleados:** control de horarios, turnos y costos de nómina asociados.
- **Módulo de gastos:** registro de gastos operativos del negocio (arriendo, servicios, transporte, etc.).

El producto esperado es una aplicación web/escritorio que permita al dueño (o encargado) del negocio tener, en un solo lugar, el estado real del inventario, las finanzas y el personal, sin depender de cuadernos o Excel dispersos.

C. Justificación

Este proyecto vale la pena porque ataca un problema real y actual de la empresa familiar, que actualmente no cuenta con control centralizado de su inventario ni de sus finanzas; al manejar múltiples categorías de producto con dinámicas distintas víveres y mecatos con alta rotación, licores y cigarrillos de alto valor unitario, gaseosas y jugos perecederos, la falta de un sistema adecuado se traduce directamente en pérdidas económicas por vencimientos, mal conteo o robo hormiga, mientras que un ERP propio permitiría tomar decisiones basadas en datos reales sobre qué se vende más, qué se debe reabastecer y en qué se está gastando de más, siendo además una solución escalable que puede iniciar con lo esencial (inventario, ventas y gastos) e ir incorporando módulos adicionales, como facturación electrónica, más adelante.

D. Usuario final

- **Dueños del negocio:** para ver el panorama general del negocio (ventas, gastos, ganancias).
- **Empleados/encargados de caja o bodega:** para registrar ventas y movimientos de inventario en el día a día.
- Opcionalmente, a futuro: **proveedores**, si se integra un módulo de pedidos automáticos según stock mínimo.

E. Utilidad (retorno de inversión)

El retorno de inversión se logra mediante:

- **Reducción de pérdidas**, ya que productos como licores y cigarrillos (de alto valor) o perecederos (jugos, gaseosas) son fáciles de perder por mal conteo o vencimiento sin control.
- **Mejor negociación con proveedores**, al saber con certeza qué y cuánto se necesita comprar, evitando sobrestock o desabastecimiento.
- **Optimización del tiempo del personal**, que dejaría de hacer conteos manuales o llevar cuentas en papel.

- **Visibilidad financiera real**, permitiendo saber si el negocio está siendo rentable categoría por categoría (por ejemplo, si los licores dejan más margen que los mecatos, y ajustar estrategia de compra).

3. Espacio de trabajo

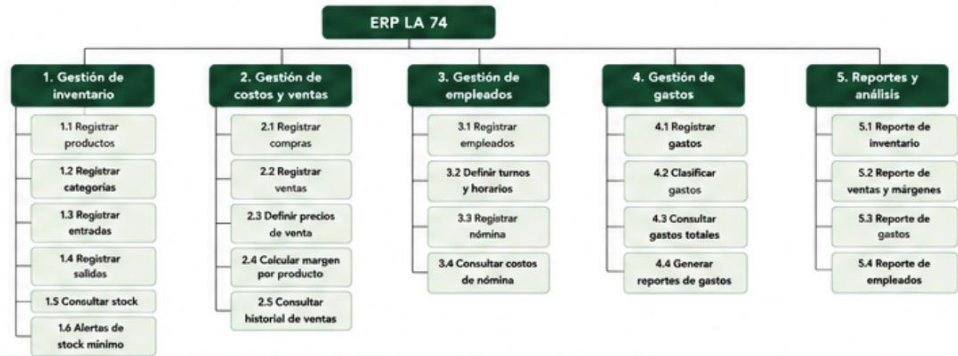
- **Repositorio en Github:** <https://github.com/jcamiloces/ERP-Software-Architecture-2026.git>
- **Pagina Web (Github Pages):** <https://jcamiloces.github.io/ERP-Software-Architecture-2026/>

4. Requerimientos.

A. ANTECEDENTES						
Cada miembro del equipo investiga aplicaciones similares a la que se pretende desarrollar. A continuación, se presenta un cuadro comparativo con una descripción de las aplicaciones consultadas y las funcionalidades que ofrecen.						
Aplicación (Dir web)	COSTO	MÓD. INVENTARIO	MÓD. COSTOS/VENTAS	MÓD. EMPLEADOS/ NÓMINA	MÓD. GASTOS	REPORTES Y ANALÍTICAS
 www.siigo.com	Planes de pago mensual	✓	✓	✓	✓	✓
 www.alegra.com	Planes de pago mensual	✓	✓	✓	✓	✓
 www.sap.com	Licencia + mantenimiento (alto costo)	✓	✓	✓	✓	✓
 LA 74 ERP (Proyecto)	Desarrollo propio (bajo costo)	✓	✓	✓	✓	✓
Descripción de las aplicaciones consultadas • Siigo Nube: ERP en la nube que permite gestionar inventarios, compras, ventas, gastos, nómina y contabilidad, con múltiples reportes financieros. • Alegra: Solución en la nube para PYMES, que integra inventario, facturación, gastos, banca y nómina. • SAP Business One: ERP empresarial robusto para medianas empresas, con amplio alcance funcional y soporte, pero con costos elevados de licencia y mantenimiento. • LA 74 ERP (propuesta): Sistema a la medida para la empresa Viveres y Licores la 74, enfocado en inventario			Valor agregado de nuestro proyecto  Control de inventario por categorías (cigarrillos, licores, viveres, etc.).  Alertas de stock mínimo y productos por vencer.  Cálculo de margen por producto y por categoría.  Control de empleados y costos			

B. REQUISITOS FUNCIONALES

A continuación, se presentan los requisitos funcionales del sistema a través de un árbol de descomposición funcional (máximo 4 niveles de profundidad) y la lista de requisitos principales.



REQUISITOS FUNCIONALES PRINCIPALES			
Código	Requisito	Código	Requisito
RF01	El sistema deberá permitir registrar productos por categoría.	RF08	El sistema deberá calcular y registrar la nómina de empleados.
RF02	El sistema deberá registrar entradas y salidas de inventario.	RF09	El sistema deberá registrar y clasificar los gastos del negocio.
RF03	El sistema deberá generar alertas cuando el stock esté por debajo del mínimo definido.	RF10	El sistema deberá mostrar reportes de inventario.
RF04	El sistema deberá registrar compras a proveedores.	RF11	El sistema deberá mostrar reportes de ventas y márgenes.
RF05	El sistema deberá registrar ventas de productos.	RF12	El sistema deberá mostrar reportes de gastos.
RF06	El sistema deberá calcular el margen de ganancia por producto y por categoría.	RF13	El sistema deberá mostrar reportes de empleados y costos de nómina.
RF07	El sistema deberá gestionar empleados, turnos y horarios.	RF14	El sistema deberá permitir gestionar usuarios y permisos.

C. REQUISITOS NO FUNCIONALES

Menciona cuáles son los requisitos no funcionales más relevantes para el contexto del proyecto. Justifica tu respuesta.

Requisito no funcional	Descripción	Justificación
 1. Usabilidad	Interfaz sencilla e intuitiva para que empleados con pocos conocimientos técnicos puedan usarla fácilmente.	Facilita el registro de ventas e inventario en el día a día y reduce errores operativos.
 2. Seguridad	Autenticación de usuarios y control de permisos por roles (administrador, empleado, cajero, bodega).	Protege la información financiera y del inventario del negocio.
 3. Rendimiento	Las operaciones comunes (ventas, consulta de stock, reportes) deberán ejecutarse en pocos segundos.	Evita demoras en la atención al cliente y mejora la productividad.
 4. Disponibilidad	El sistema debe estar disponible durante el horario de funcionamiento y contar con copias de respaldo.	Evita interrupciones en el negocio y pérdidas de información crítica.
 5. Escalabilidad	El sistema debe permitir agregar nuevas categorías, usuarios y módulos sin afectar su funcionamiento.	El negocio puede crecer y el sistema debe adaptarse a nuevas necesidades.
 6. Compatibilidad	Debe funcionar correctamente en navegadores modernos y en dispositivos móviles.	Permite acceder al sistema desde diferentes equipos y dispositivos.
 7. Mantenibilidad	Código organizado y documentado para facilitar actualizaciones y correcciones futuras.	Reduce costos de mantenimiento y permite mejoras continuas.
 8. Diseño de interfaz	Diseño moderno y consistente con la identidad visual del negocio (verde esmeralda, dorado, blanco y negro).	Refuerza la identidad de la marca y mejora la experiencia del usuario.

- **Alcances del Sistema**

A continuación, se describen las limitaciones y alcances del desarrollo según los objetivos previamente establecidos, considerando la triple restricción (alcance, tiempo y costo).

- Alcance funcional (qué incluye)
 - Módulo de inventario: registro de productos, categorías, entradas, salidas, consulta de stock y alertas de stock mínimo.
 - Módulo de costos y ventas: registro de compras, ventas, definición de precios y cálculo de margen por producto/categoría.
 - Módulo de empleados: registro de empleados, turnos, horarios y cálculo de nómina.
 - Módulo de gastos: registro y clasificación de gastos operativos, y consulta de gastos totales.
 - Módulo de reportes: generación de reportes de inventario, ventas, gastos y empleados.
 - Gestión básica de usuarios y permisos por rol (administrador, cajero, bodega).
- Fuera de alcance (qué no incluye, por ahora)
 - Facturación electrónica ante la DIAN u otra entidad fiscal.
 - Integración con pasarelas de pago o plataformas de e-commerce.
 - Módulo de pedidos automáticos a proveedores (queda como trabajo futuro).
 - Soporte multi-sucursal o multi-negocio.
 - Aplicación móvil nativa (solo se garantiza compatibilidad web responsive).
 - Migración de datos históricos desde Excel u otros sistemas.
- Restricciones (triple restricción)
 - Tiempo: El desarrollo está limitado al calendario académico del curso de Arquitectura de Software (un semestre), lo cual condiciona la cantidad de módulos que se pueden completar e integrar.
 - Costo: Al ser un proyecto académico/propio, no se cuenta con presupuesto para licencias, hosting premium o servicios pagos; se utilizan herramientas gratuitas y de código abierto.
 - Alcance: El sistema se limita a las funcionalidades esenciales (inventario, ventas, costos, gastos y empleados) definidas en los requerimientos, dejando funcionalidades avanzadas como trabajo futuro.

- **Tecnologías Seleccionadas**

A continuación, se mencionan los entornos de desarrollo, plataformas y herramientas empleadas en la implementación del sistema.

Categoría	Tecnología	Uso
Backend	Laravel (PHP)	Framework principal del sistema, siguiendo el patrón MVC (Modelo-Vista-Controlador)
Base de datos	MySQL	Almacenamiento de la información de inventario, ventas, empleados y gastos.
Frontend	Bootstrap	Framework CSS para una interfaz responsive, consistente con la identidad visual del negocio (verde esmeralda, dorado, blanco y negro).
Control de versiones	Git y GitHub	Control de versiones del código fuente y trabajo colaborativo en equipo.
Documentación	Markdown / arc42 / PlantUML	Documentación técnica del proyecto (definición, requerimientos, arquitectura).
Gestión ágil	Jira	Gestión del backlog y planificación de sprints.

Todas las herramientas seleccionadas son de código abierto o de uso gratuito, lo cual es coherente con la restricción de costo del proyecto y con el hecho de tratarse de un desarrollo propio de bajo costo frente a soluciones comerciales como Siigo, Alegra o SAP Business One.

Autonomous work activity

The Strengths and Weaknesses of Software Architecture Design in the RUP, MSF, MBASE and RUP-SOA Methodologies: A Conceptual Review



ABOUT THE ARTICLE

This conceptual review analyzes and compares how four methodologies address software architecture design: RUP, MSF, MBASE and RUP-SOA. It identifies their main strengths and weaknesses to provide guidance on choosing the most suitable approach depending on project needs.



EVALUATION CRITERIA

- ✓ Architecture focus and activities
- ✓ Integration with the development process
- ✓ Support for quality attributes
- ✓ Flexibility and adaptability
- ✓ Scalability and complexity handling
- ✓ Documentation and guidelines

RUP Rational Unified Process	MSF Microsoft Solutions Framework	MBASE Model-Based Architecture for Software Engineering	RUP-SOA RUP for Service-Oriented Architecture
A phased, iterative process framework that emphasizes architecture early and throughout the project.	A process model from Microsoft that provides guidance for building and deploying solutions.	A methodology centered on architectural models as primary artifacts.	An adaptation of RUP to guide the development of Service-Oriented Architectures (SOA).
STRENGTHS	STRENGTHS	STRENGTHS	STRENGTHS
✓ Architecture-centric: Architecture is an explicit discipline and is addressed early in the process (Inception). ✓ Iterative and incremental: Allows architecture to evolve and be refined in each iteration. ✓ Well-structured: Clear phases (Inception, Elaboration, Construction, Transition) and disciplines. ✓ Strong tooling support: Good support from Rational tools. ✓ Risk-driven: Early identification and mitigation of architectural risks.	✓ Simplicity and practicality: Easier to understand and adopt. ✓ Well-integrated with Microsoft technologies and tools. ✓ Focus on the entire lifecycle: From vision to deployment and operations. ✓ Good guidance for teams: Roles, templates and checklists are well defined. ✓ Promotes best practices: Encourages quality and reuse.	✓ Architecture-driven: Architecture models guide the whole development. ✓ High emphasis on modeling: Promotes analysis, simulation and validation. ✓ Early validation: Models help detect issues early. ✓ Good for complex systems: Handles complexity through abstraction. ✓ Improves communication: Models are easy to understand and share.	✓ SOA focus: Addresses specific activities for SOA development. ✓ Reuse and interoperability: Promotes service reuse and loose coupling. ✓ Alignment with RUP: Leverages the strengths of RUP with SOA principles. ✓ Scalability: Well-suited for distributed and enterprise systems. ✓ Considers non-functional requirements of services.
WEAKNESSES	WEAKNESSES	WEAKNESSES	WEAKNESSES
✗ Complex and heavy: Can be overkill for small or simple projects. ✗ Steep learning curve: Requires skilled roles and experience. ✗ Documentation intensive: May generate a large amount of documentation. ✗ Less flexible in practice: Tailoring is needed but not always easy.	✗ Less explicit on architecture: Architecture activities are not as deep or prescriptive as in RUP. ✗ Technology-dependent: Strongly tied to Microsoft stack. ✗ Limited flexibility: Customization can be limited. ✗ Not ideal for highly complex or large-scale systems.	✗ Modeling effort: Requires significant time and expertise in modeling. ✗ Tool support can be limited or expensive. ✗ Not widely adopted: Less community and industrial use. ✗ May be excessive for small projects.	✗ Complexity: Adds SOA complexity on top of RUP. ✗ Requires SOA expertise: Specialized knowledge is needed. ✗ Tooling and standards dependency. ✗ Not necessary for projects that do not require SOA.

COMPARATIVE SUMMARY

CRITERIA	RUP	MSF	MBASE	RUP-SOA
Architecture Focus	High	Medium	Very High	High (SOA)
Ease of Adoption	Medium	High	Low	Medium
Flexibility	Medium	Medium-Low	Medium	Medium
Support for Quality Attributes	High	Medium	High	High
Scalability for Large Systems	High	Medium	High	Very High
Tool Support	High	High (Microsoft)	Medium	Medium



KEY CONCLUSIONS

- RUP is a robust and complete framework for architecture, ideal for medium to large projects with high quality requirements.
- MSF is practical and easy to adopt, especially in environments using Microsoft technologies.
- MBASE provides a strong architectural foundation through modeling, suitable for complex and critical systems.
- RUP-SOA extends RUP to address the challenges of SOA, enabling the development of flexible, reusable and interoperable systems.



TAKEAWAY

There is no "one-size-fits-all" methodology. The choice depends on the project's context, team skills, technology, and quality and architecture goals.



Source: The strengths and weaknesses of software architecture design in the RUP, MSF, MBASE and RUP-SOA methodologies: A conceptual review. (ScienceDirect)